whereas the forward and inverse model losses share a scaling parameter, β, so that they're inversely related. This is the only loss function we backpropagate through, so at each training step we backpropagate through all four models starting from this single loss function.

The `max_episode_len` and `min_progress` parameters are used to set the minimum amount of forward progress Mario must make or we'll reset the environment. Sometimes Mario will get stuck behind an obstacle and will just keep taking the same action forever, so if Mario doesn't move forward enough in a reasonable amount of time, we just assume he's stuck.

During training, if the policy function says to take action 3 (for example), we will repeat that action six times (set according to the `action_repeats` parameter) instead of just once. This helps the DQN learn the value of actions more quickly. During testing (i.e., inference), we only take the action once. The gamma parameter is the same gamma parameter from the DQN chapter. When training the DQN, the target value is not just the current reward r_t but the highest predicted action value for the next state, so the full target is $r_t + \gamma \times \max(Q(S_{t+1}))$. Lastly, the `frames_per_state` parameter is set to `3` since each state is the last three frames of the game play.

Listing 8.9. The loss function and reset environment

```
def loss_fn(q_loss, inverse_loss, forward_loss):
    loss_ = (1 - params['beta']) * inverse_loss
    loss_ += params['beta'] * forward_loss
    loss_ = loss_.sum() / loss_.flatten().shape[0]
    loss = loss_ + params['lambda'] * q_loss
    return loss

def reset_env():
    """
    Reset the environment and return a new initial state
    """
    env.reset()
    state1 = prepare_initial_state(env.render('rgb_array'))
    return state1
```

Finally, we get to the actual ICM function.